

PYTHON MASTER STUDY BOOK

A Complete Textbook for Programming with Python (Course Code
2501IT42)

VOLUME 5

UNIT V — Errors, Exceptions and Graphical User Interfaces

The final volume: a clear, visual treatment of error handling with the full try/except/else/finally flow diagrammed, followed by a hands-on introduction to building simple GUI programs with Tkinter.

UNIT V

Errors, Exceptions and Graphical User Interfaces

Learning Objectives (CO5)

- Distinguish syntax errors, runtime errors, and logical errors.
- Handle exceptions gracefully using try/except/else/finally.
- Raise exceptions deliberately, and create your own custom exception classes.
- Understand cleanup actions and why `with` is the safest cleanup pattern.
- Understand the difference between terminal-based and GUI-based programs.
- Build a simple working GUI application using Tkinter: windows, labels, buttons, and entry fields.

Prerequisites

Units I-IV — especially functions, classes (custom exceptions are classes), and file handling (many exceptions in this unit come directly from working with files).

Part A — Errors and Exceptions

1. Types of Errors: Syntax, Runtime, and Logical

Not all mistakes in a program are the same kind of problem. It helps enormously, especially in exams, to correctly name which type of error you're looking at.

Type	What it means	Example	When it's caught
Syntax error	Code breaks Python's grammar rules -- it isn't even valid Python.	<code>print("hi"</code>	BEFORE the program runs at all
Runtime error (exception)	Code is grammatically valid, but something goes wrong while it is RUNNING.	<code>10 / 0</code>	WHILE the program is running
Logical error	Code runs with no error message at all, but produces the WRONG result because the logic itself is flawed.	Using <code>+</code> instead of <code>*</code> in an area formula	Never automatically -- you must test and debug to find it

COMPETITIVE EXAM / GATE FOCUS

Logical errors are by far the hardest to catch, precisely because Python gives no error message at all -- the program runs 'successfully' and simply gives a wrong answer. This is exactly why the 'Test' step of the Program Development Cycle (Unit I) is essential: you must check the OUTPUT against what you expected, not just check that the program ran without crashing.

An **exception** is Python's way of representing a runtime error as an object. When something goes wrong while running, Python 'raises' (throws) an exception object describing what happened. If nothing catches it, the program stops and Python prints a **traceback** (the error message you see, showing where the error occurred).

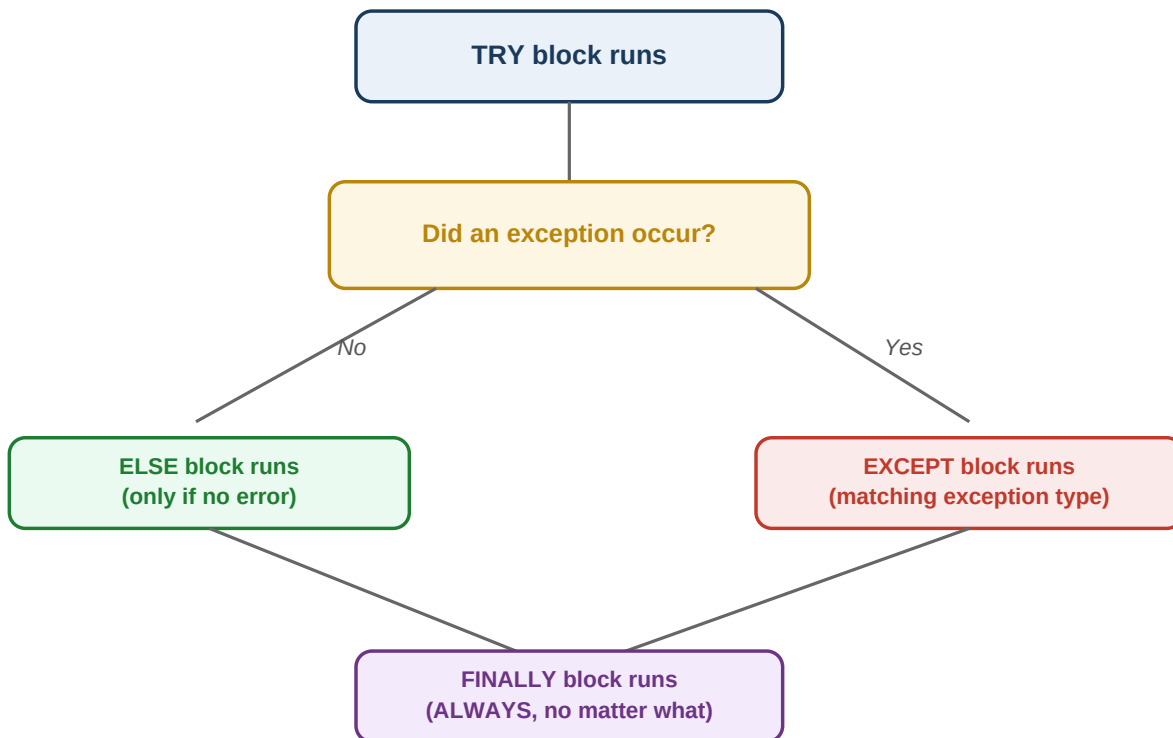
```
print(10 / 0)
# Traceback (most recent call last):
#   File "example.py", line 1, in <module>
#     print(10 / 0)
# ZeroDivisionError: division by zero
```

Common exception	When it happens
ZeroDivisionError	Dividing any number by 0
ValueError	A function receives a value of the right TYPE but the wrong VALUE, e.g. <code>int("abc")</code>
TypeError	An operation is applied to an incompatible type, e.g. <code>"5" + 5</code>
IndexError	Using an out-of-range index on a list/string
KeyError	Accessing a dictionary key that doesn't exist
FileNotFoundError	Opening a file in 'r' mode that doesn't exist
NameError	Using a variable that hasn't been defined
AttributeError	Calling a method/attribute that doesn't exist on that object

2. Handling Exceptions: try, except, else, finally

Instead of letting an exception crash the program, you can CATCH it using try/except, and decide what to do instead.

The full try / except / else / finally control flow



else runs ONLY if no exception occurred; finally ALWAYS runs, no matter what happened.

```

try:
    num = int(input("Enter a number: "))
    result = 10 / num
except ZeroDivisionError:
    print("You cannot divide by zero!")
except ValueError:
    print("That was not a valid number!")
else:
    # runs ONLY if no exception happened
    print("Success! Result:", result)
finally:
    # ALWAYS runs, error or not
    print("Done attempting the calculation.")
  
```

Block	Runs when...
try	Always -- this is the code that might raise an exception.

Block	Runs when...
except	ONLY if an exception matching its type occurred in the try block.
else	ONLY if the try block completed with NO exception at all.
finally	ALWAYS -- whether or not an exception occurred, and even if the except block itself raised a new exception. Typically used for cleanup (closing files, releasing resources).

Handling Multiple Exceptions

```
try:
    value = int(input("Enter a positive number: "))
    result = 10 / value
# catch SEVERAL types in one block
except (ZeroDivisionError, ValueError) as e:
    print("Something went wrong:", e)
```

```
try:
    risky_operation()
# catches almost ANY exception -- use sparingly!
except Exception as e:
    print("Unexpected error:", e)
```

COMMON MISTAKES

- Using a bare `except :` (catching everything, no type at all) as a default habit -- this can silently hide real bugs (even typos like calling a misspelled function name) instead of surfacing them. Always catch the SPECIFIC exception type you expect when possible.
- Putting so much code inside the try block that you're not sure WHICH line could actually raise the exception -- keep try blocks focused on the risky operation only.

COMPETITIVE EXAM / GATE FOCUS

Exception matching checks `except` clauses IN ORDER, top to bottom, and stops at the first match -- exactly like `if/elif/else`. This means a more general exception type (like the base class `Exception`) placed BEFORE a more specific one (like `ValueError`) will catch everything first, making the specific `except` clause below it unreachable. Always list specific exceptions before general ones.

3. Raising Exceptions

You can deliberately trigger an exception yourself using `raise`, typically to reject invalid input or an invalid program state early, with a clear error message.

```
def set_age(age):
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

try:
    set_age(-5)
except ValueError as e:
    print("Caught:", e)      # Caught: Age cannot be negative
```

4. User-Defined (Custom) Exceptions

You can create your OWN exception types by defining a class that inherits from Python's built-in `Exception` class. This is useful when the built-in exception types (`ValueError`, `TypeError`, etc.) don't clearly describe the specific problem in YOUR program.

```
# custom exception -- inherits from Exception
class InsufficientBalanceError(Exception):
    pass

class BankAccount:
    def __init__(self, balance):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientBalanceError(
                f"Cannot withdraw {amount}, balance is only {self.balance}")
        self.balance -= amount

acc = BankAccount(100)
try:
    acc.withdraw(500)
except InsufficientBalanceError as e:
    print("Transaction failed:", e)
```

COMPETITIVE EXAM / GATE FOCUS

A custom exception class combines two units of learning at once: inheritance (Unit IV) and exception handling (this unit). This is a common exam/interview task -- being asked to design a meaningful, specific exception for a given real-world scenario, rather than reusing a generic built-in one.

5. Cleanup Actions: finally and the with Statement

A **cleanup action** is code that **MUST** run regardless of whether an error occurred -- closing a file, releasing a lock, disconnecting from a database. There are two ways to guarantee this in Python:

```
# Manual cleanup using finally
f = None
try:
    f = open("data.txt", "r")
    content = f.read()
    result = 10 / 0          # suppose an error happens here
except ZeroDivisionError:
    print("Math error occurred")
finally:
    if f:
        # this STILL runs, closing the file safely
        f.close()

# Preferred, modern cleanup using with (does the same job automatically)
try:
    with open("data.txt", "r") as f:
        content = f.read()
        result = 10 / 0
except ZeroDivisionError:
    print("Math error occurred")
# no finally needed -- 'with' already guarantees f.close() happened
```

MUST REMEMBER

- Syntax error = invalid Python grammar, caught before running. Runtime error (exception) = valid code that fails while running. Logical error = runs fine but gives the wrong answer.
- try runs the risky code; except handles a matching exception; else runs only if NO exception occurred; finally ALWAYS runs, used for cleanup.
- List specific except types before general ones -- Python matches top to bottom, first match wins.
- raise triggers an exception deliberately; custom exceptions are classes that inherit from Exception.
- `with` is the modern, safer alternative to manual try/finally for guaranteed cleanup (like closing files).

INTERVIEW QUESTIONS

Q: What is the difference between an error and an exception?

A: 'Error' is the general term for anything wrong in a program, including syntax errors that stop it from even running. An 'exception' specifically refers to a runtime problem, represented as an object, that occurs while the program is executing and can potentially be caught and handled with try/except so the program can continue running.

Q: When would you create a custom exception class instead of using a built-in one?

A: When a built-in exception (ValueError, TypeError, etc.) doesn't clearly communicate WHAT specifically went wrong in your program's own domain -- e.g. InsufficientBalanceError is far clearer to future readers and calling code than a generic ValueError raised for the same situation.

Part B — Graphical User Interfaces

1. Terminal-Based Programs vs GUI-Based Programs

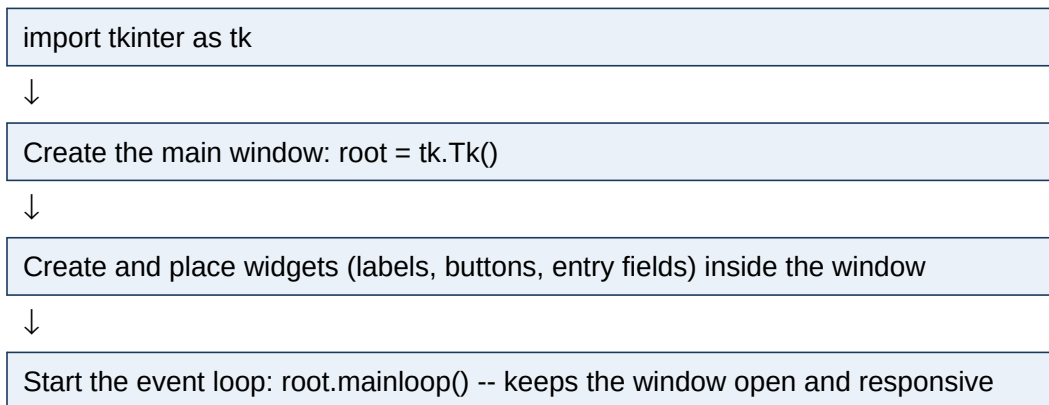
Every program you've written so far in this book is a **terminal-based** (command-line) program: it interacts with the user through typed text, using `input()` and `print()`. A **GUI-based** (Graphical User Interface) program instead shows a visual window with clickable buttons, text boxes, and other visual controls.

Terminal-based	GUI-based
Interaction is entirely through typed text	Interaction is through visual controls: buttons, text boxes, menus
Simple and fast to build	Takes more code to set up windows and widgets
No mouse interaction	Naturally supports mouse clicks and visual feedback
Good for quick scripts, automation, servers	Good for end-user-facing desktop applications

2. Introducing Tkinter

Tkinter is Python's built-in standard library module for building simple GUI applications -- no separate installation is needed. Every Tkinter program follows the same basic shape:

The standard shape of every Tkinter program



```
import tkinter as tk
```

```
root = tk.Tk()                                # create the main window
root.title("My First GUI App")                 # set the window title
# set window size: width x height
root.geometry("300x150")
```

```
# create a label widget
label = tk.Label(root, text="Hello, GUI!")
# pack() places it in the window
label.pack()

# keeps the window open, listening for events
root.mainloop()
```

What this program produces



3. Common Widgets: Labels, Buttons, Entry Fields

Widget	Purpose	Create with
Label	Displays text or an image (not editable by the user)	<code>tk.Label(root, text="...")</code>
Button	A clickable button that triggers an action	<code>tk.Button(root, text="...", command=func)</code>
Entry	A single-line text box for user input	<code>tk.Entry(root)</code>
Text	A multi-line text box	<code>tk.Text(root)</code>

4. Events: Connecting a Button to a Function

An **event** is something that happens while the program runs, like a button click. Tkinter lets you connect a widget to a function using the `command` parameter (for buttons) — that function runs automatically whenever the event happens.

```
import tkinter as tk

# this function runs when the button is clicked
def greet():
    # get() reads the current text typed in the entry field
    name = entry.get()
    # config() updates a widget after it is created
    label.config(text=f"Hello, {name}!")

root = tk.Tk()
```

```
root.title("Greeter")

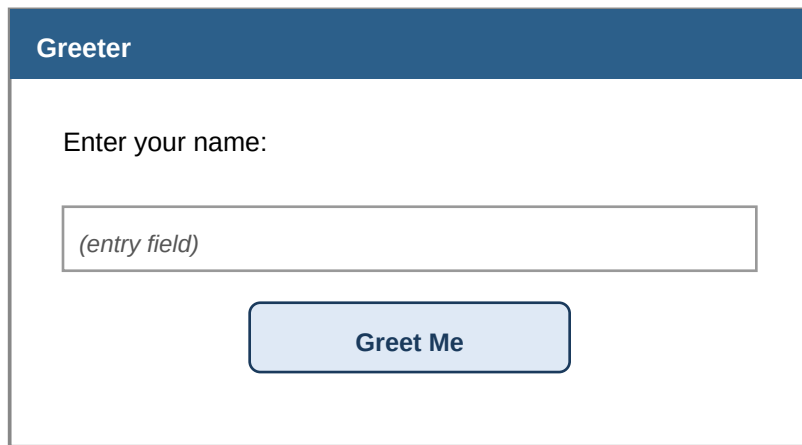
label = tk.Label(root, text="Enter your name:")
label.pack()

entry = tk.Entry(root)
entry.pack()

# command= links button to greet()
button = tk.Button(root, text="Greet Me", command=greet)
button.pack()

root.mainloop()
```

What this program looks like



Clicking 'Greet Me' calls greet(), which reads the Entry text and updates the Label.

COMPETITIVE EXAM / GATE FOCUS

The pattern of reading input with `.get()` and updating a widget with `.config(text=...)` is the single most common Tkinter pattern in exam questions and beginner GUI projects -- almost every simple Tkinter app is some variation of 'read something from an Entry, do something with it, show the result in a Label'.

5. File-Open Dialog and Displaying File Content

Tkinter includes a `filedialog` module that opens the operating system's native 'Open File' window, letting the user pick a file visually instead of typing its exact path.

```
import tkinter as tk
from tkinter import filedialog

def open_file():
    # opens the native file-picker dialog
    filepath = filedialog.askopenfilename()
    # empty string if user cancelled
    if filepath:
        with open(filepath, "r") as f:
            content = f.read()
        # clear any previous content
        text_box.delete("1.0", tk.END)
        # display the file's content
        text_box.insert(tk.END, content)

root = tk.Tk()
root.title("Simple File Viewer")

button = tk.Button(root, text="Open File", command=open_file)
button.pack()

text_box = tk.Text(root, height=10, width=40)
text_box.pack()

root.mainloop()
```

This program combines EVERYTHING from this book: a GUI window (this unit), reading a file with `with`` (Unit IV), and a button connected to a function via an event (this unit) — a genuinely complete, practical mini-application.

QUICK REVISION

- Terminal programs use `input()/print()`; GUI programs use visual windows and widgets.
- Every Tkinter program: create `root = tk.Tk()`, add widgets, call `root.mainloop()`.
- Common widgets: Label (display text), Entry (single-line input), Button (clickable, runs a function via `command=`), Text (multi-line box).
- `.get()` reads an Entry's current text; `.config(text=...)` updates a widget's displayed text/content.
- `filedialog.askopenfilename()` opens a native file picker, returning the chosen file's path.

Practice Problems (Unit V)

A. Beginner

E1. Python Try/Except -- Handle Division by Zero (syllabus: HackerRank py-exceptions)

TRY YOURSELF

E1. Python Try/Except -- Handle Division by Zero (syllabus: HackerRank py-exceptions)

Read two integers and print their division; handle the case where the divisor is zero.

Input: 10, 0

Output: Error Code: integer division or modulo by zero

HINT

Wrap the division in a try block; catch ZeroDivisionError specifically.

SOLUTION

See code and explanation below.

```
a = int(input())
b = int(input())
try:
    print(a // b)
except ZeroDivisionError as e:
    print("Error Code:", e)
```

Catching ZeroDivisionError specifically (rather than a bare except:) means only THIS particular problem is handled gracefully, while any other unexpected error would still surface normally, which is exactly the safer habit this chapter recommends.

E2. Handling Multiple Exceptions (syllabus: HackerRank handling-multiple-exceptions)

TRY YOURSELF**E2. Handling Multiple Exceptions (syllabus: HackerRank handling-multiple-exceptions)**

Read two integers and divide them, correctly handling BOTH a ValueError (bad input) and a ZeroDivisionError (dividing by zero).

Input: 'abc', 5

Output: Error Code: invalid literal for int() with base 10: 'abc'

HINT

Use two separate except clauses, one per exception type, or a single tuple of types if the handling is identical.

SOLUTION

See code and explanation below.

```
try:
    a = int(input())
    b = int(input())
    print(a // b)
except ZeroDivisionError as e:
    print("Error Code:", e)
except ValueError as e:
    print("Error Code:", e)
```

Two separate except clauses let each problem get its own specific, clear message, rather than one generic 'something went wrong' message that would make debugging harder.

B. Easy**E3. Show File Handling Using try/finally and with (syllabus: CodeChef VALIDMIN)**

TRY YOURSELF**E3. Show File Handling Using try/finally and with (syllabus: CodeChef VALIDMIN)**

Practice problem listed in syllabus — read a number from a file and validate that it is above a minimum value, using proper cleanup regardless of errors.

Input: file contains: 5, minimum=10

Output: Value 5 is below the minimum of 10

HINT

Use `with` to guarantee the file closes, and raise a ValueError if the check fails.

SOLUTION

See code and explanation below.

```
def check_minimum(filepath, minimum):
    with open(filepath, "r") as f:
        value = int(f.read().strip())
    if value < minimum:
        raise ValueError(f"Value {value} is below the minimum of {minimum}")
    return value

try:
    check_minimum("data.txt", 10)
except ValueError as e:
    print(e)
except FileNotFoundError:
    print("File not found!")
```

Using `with` means the file is guaranteed to close even if the ValueError is raised right after reading it -- this is exactly why `with` is preferred over manual open()/close() when combined with error handling.

E4. Build a File-Open Dialog and Display Content (syllabus: CodeChef ERROR404, adapted for GUI)

TRY YOURSELF**E4. Build a File-Open Dialog and Display Content (syllabus: CodeChef ERROR404, adapted for GUI)**

Practice problem listed in syllabus — build a simple Tkinter app with a button that opens a file and displays its content in a text box, handling the case where the file can't be read.

Input: (GUI interaction)

Output: File content displayed in the text box, or a friendly error message

HINT

Wrap the file-reading part in a try/except inside the button's callback function, so a bad file doesn't crash the whole GUI app.

SOLUTION

See code and explanation below.

```
import tkinter as tk
from tkinter import filedialog, messagebox

def open_file():
    filepath = filedialog.askopenfilename()
    if not filepath:
        return
    try:
        with open(filepath, "r") as f:
            content = f.read()
            text_box.delete("1.0", tk.END)
            text_box.insert(tk.END, content)
    except Exception as e:
        messagebox.showerror("Error", f"Could not open file: {e}")

root = tk.Tk()
root.title("File Viewer")
tk.Button(root, text="Open File", command=open_file).pack()
text_box = tk.Text(root, height=10, width=40)
text_box.pack()
```

```
root.mainloop()
```

This ties the whole book together: GUI widgets and events (this unit), file reading with `with` (Unit IV), and exception handling (this unit) all in one small, genuinely useful program. messagebox.showerror() is another Tkinter helper that pops up a native error dialog.`

C. Interview-style

E5. Design a Custom Exception for Invalid Input (original problem)

TRY YOURSELF

E5. Design a Custom Exception for Invalid Input (original problem)

Write a function that validates an age (must be between 0 and 120), raising a custom `InvalidAgeError` with a clear message if it isn't.

Input: age=150

Output: `InvalidAgeError: Age must be between 0 and 120, got 150`

HINT

Define a custom exception class inheriting from `Exception`, then raise it with a clear message when the check fails.

SOLUTION

See code and explanation below.

```
class InvalidAgeError(Exception):
    pass

def validate_age(age):
    if not (0 <= age <= 120):
        raise InvalidAgeError(f"Age must be between 0 and 120, got {age}")
    return age

try:
    validate_age(150)
```

```
except InvalidAgeError as e:  
    print(f"InvalidAgeError: {e}")
```

A custom exception like this documents the VALID rule (0 to 120) directly in the code and gives any caller a specific, catchable exception type to handle -- a common real-world validation pattern that interviewers like to see.

Master Comparison Tables (Unit V)

Error vs Exception

Error	Exception
General term for anything wrong (includes syntax errors that stop the program before it even runs)	Specifically a RUNTIME problem, represented as an object, that CAN be caught with try/except

Syntax Error vs Runtime Error vs Logical Error

Syntax error	Runtime error	Logical error
Invalid Python grammar; caught before running	Valid code fails while running (an exception)	Runs fine, wrong answer -- no error message at all

try/except vs try/finally

try/except	try/finally
Catches and HANDLES a specific exception if one occurs, letting the program continue	ALWAYS runs its block, whether or not an exception occurred -- used for guaranteed cleanup
Can be combined with finally in the same try statement	Can be combined with except in the same try statement

Unit V Mock Test

- Which is a syntax error?
(a) 10/0 (b) print("hi") (c) int("abc") (d) x[10] on a short list
- What does the finally block guarantee?
(a) it only runs on success (b) it only runs on failure (c) it ALWAYS runs (d) it never runs
- Which exception occurs from int("abc")?
(a) TypeError (b) ValueError (c) NameError (d) KeyError
- What does raise do?
(a) catches an exception (b) deliberately triggers an exception (c) closes a file (d) defines a function

5. A custom exception class should inherit from:
(a) object only (b) Exception (c) BaseError (d) dict
6. What does else do in try/except/else?
(a) runs if an exception occurred (b) runs only if NO exception occurred (c) always runs
(d) never runs
7. Tkinter's mainloop() is needed to:
(a) close the window (b) keep the window open and responsive to events (c) read a file
(d) raise an exception
8. Which widget is used for a single-line text input?
(a) Label (b) Button (c) Entry (d) Text
9. .get() on an Entry widget:
(a) sets its text (b) reads its current text (c) deletes it (d) closes the window
10. Which error type describes a bug that runs with no crash but gives the wrong answer?
(a) syntax error (b) runtime error (c) logical error (d) NameError

Coding Questions

- Write a function that safely converts a string to an integer, returning None and printing a friendly message if the conversion fails.
- Write a program that reads numbers from a file and prints their sum, using try/except/finally to handle a missing file gracefully.
- Create a custom exception NegativeValueError and use it to validate a list of numbers, raising it for any negative number found.
- Build a simple Tkinter calculator with two Entry fields, a Button, and a Label showing the sum when clicked.
- Write a program with nested try/except blocks: an outer block catches a general Exception, and an inner block specifically catches ZeroDivisionError.

Answer Key

Q	Answer	Q	Answer
1	(b) print("hi" -- missing closing parenthesis	6	(b) runs only if NO exception occurred

Q	Answer	Q	Answer
2	(c) it ALWAYS runs	7	(b) keeps the window open and responsive
3	(b) ValueError -- valid type, invalid value	8	(c) Entry -- single-line text input
4	(b) deliberately triggers an exception	9	(b) reads its current text
5	(b) Exception	10	(c) logical error

This completes all five units of the Python Master Study Book. A combined final volume with the master roadmap, complete cheat sheets, and a full cross-unit question bank can be assembled next if useful.